Session 5 - Pandas

In [1]:
```
import pandas
```

In [42]:
```
station_ids = pandas.read_csv("data/Niederschlag_1981-2010_Stationsliste.txt
                              encoding="iso-8859-1", delimiter=" *; *", #sta
                             index_col="Stations_id", engine="python") #read
```

In [43]:
```
station_ids #show an overview
```

Out[43]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bund
1	Aach	47.841299	8.849299	478.0	Württ
2	Aachen (Kläranlage)	50.806581	6.099638	138.0	No W
3	Aachen	50.782659	6.094087	202.0	No W
4	Aachen-Brand	50.768313	6.120705	243.0	No W
6	Aalen-Unterrombach	48.836072	10.059764	455.0	Württ
...	...	...	...	...	
19697	Wassertrüdingen/Wörnitz	49.046341	10.602684	435.0	
19698	Werda/Vogtland	50.448020	12.305168	595.0	S
19701	Wertingen/Zusam	48.557500	10.674167	430.0	
19706	Oberndorf am Neckar	48.285077	8.576395	516.0	Württ
19781	Ingolstadt	48.742909	11.423324	367.0	

5740 rows × 6 columns

In [44]:
```
station_ids.columns #tell us which columns are there
```

Out[44]:
```
Index(['Stationsname', 'geogr. Breite', 'geogr. Laenge', 'Stationshoehe',
       'Bundesland', 'Unnamed: 6'],
      dtype='str')
```

In [45]:
```
station_ids.info() #get information; brackets to display it nicely
```

```
<class 'pandas.DataFrame'>
Index: 5740 entries, 1 to 19781
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Stationsname   5740 non-null   str
 1   geogr. Breite  5740 non-null   float64
 2   geogr. Laenge  5740 non-null   float64
 3   Stationshoehe  5740 non-null   float64
 4   Bundesland     5740 non-null   str
 5   Unnamed: 6     0 non-null      float64
dtypes: float64(4), str(2)
memory usage: 313.9 KB
```

In [46]:
```
station_ids.head() #show first five lines
```

Out[46]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
1	Aach	47.841299	8.849299	478.0	Baden-Württemberg
2	Aachen (Kläranlage)	50.806581	6.099638	138.0	Nordrhein-Westfalen
3	Aachen	50.782659	6.094087	202.0	Nordrhein-Westfalen
4	Aachen-Brand	50.768313	6.120705	243.0	Nordrhein-Westfalen
6	Aalen-Unterrombach	48.836072	10.059764	455.0	Baden-Württemberg

In [47]:
```
station_ids.tail() #show last five lines
```

Out[47]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bund
19697	Wassertrüdingen/Wörnitz	49.046341	10.602684	435.0	
19698	Werda/Vogtland	50.448020	12.305168	595.0	S
19701	Wertingen/Zusam	48.557500	10.674167	430.0	
19706	Oberndorf am Neckar	48.285077	8.576395	516.0	Württ
19781	Ingolstadt	48.742909	11.423324	367.0	

In [48]:
```
station_ids.drop (station_ids.columns [-1], axis=1, inplace=True) #delete th
```

In [49]:
```
station_ids.columns # run to check the working of prior line
```

Out[49]:
```
Index(['Stationsname', 'geogr. Breite', 'geogr. Laenge', 'Stationshoehe',
       'Bundesland'],
      dtype='str')
```

In [50]:
```
station_ids.info()
```

```
<class 'pandas.DataFrame'>
Index: 5740 entries, 1 to 19781
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Stationsname   5740 non-null   str
 1   geogr. Breite  5740 non-null   float64
 2   geogr. Laenge  5740 non-null   float64
 3   Stationshoehe  5740 non-null   float64
 4   Bundesland     5740 non-null   str
dtypes: float64(3), str(2)
memory usage: 269.1 KB
```

In [51]:
```
station_ids.head()
```

Out[51]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
1	Aach	47.841299	8.849299	478.0	Baden-Württemberg
2	Aachen (Kläranlage)	50.806581	6.099638	138.0	Nordrhein-Westfalen
3	Aachen	50.782659	6.094087	202.0	Nordrhein-Westfalen
4	Aachen-Brand	50.768313	6.120705	243.0	Nordrhein-Westfalen
6	Aalen-Unterrombach	48.836072	10.059764	455.0	Baden-Württemberg

In [52]:
```
station_ids.iloc[2,0] #iloc: use the index 2,0 (see loading of data, which m
```

Out[52]:
```
'Aachen'
```

In [53]:
```
station_ids.iloc[:5] #slice 5 out of it
```

Out[53]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
1	Aach	47.841299	8.849299	478.0	Baden-Württemberg
2	Aachen (Kläranlage)	50.806581	6.099638	138.0	Nordrhein-Westfalen
3	Aachen	50.782659	6.094087	202.0	Nordrhein-Westfalen
4	Aachen-Brand	50.768313	6.120705	243.0	Nordrhein-Westfalen
6	Aalen-Unterrombach	48.836072	10.059764	455.0	Baden-Württemberg

In [54]:
```
station_ids.loc[:1:3]
```

Out[54]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
1	Aach	47.841299	8.849299	478.0	Baden-Württemberg

In [55]:
```
station_ids.loc[3,"Stationsname"]
```

Out[55]:
```
'Aachen'
```

In [56]:
```
station_ids.loc[3:6,"Stationsname"] # after slicing 5 is missing nows
```

Out[56]:
```
Stations_id
3                Aachen
4           Aachen-Brand
6    Aalen-Unterrombach
Name: Stationsname, dtype: str
```

In [57]:
```
station_ids.loc["geogr. Breite":"geogr. Laenge"]
```

Out[57]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland

In [58]:
```
station_ids["Bundesland"]
```

Out[58]:
```
Stations_id
1          Baden-Württemberg
2        Nordrhein-Westfalen
3        Nordrhein-Westfalen
4        Nordrhein-Westfalen
6          Baden-Württemberg
               ...
19697            Bayern
19698           Sachsen
19701            Bayern
19706      Baden-Württemberg
19781            Bayern
Name: Bundesland, Length: 5740, dtype: str
```

In [59]:
```
station_ids.Bundesland #for names without space also possible
```

Out[59]:
```
Stations_id
1          Baden-Württemberg
2        Nordrhein-Westfalen
3        Nordrhein-Westfalen
4        Nordrhein-Westfalen
6          Baden-Württemberg
               ...
19697            Bayern
19698           Sachsen
19701            Bayern
19706      Baden-Württemberg
19781            Bayern
Name: Bundesland, Length: 5740, dtype: str
```

In [60]:
```
station_ids.groupby("Bundesland").count().sort_values("Stationsname")
# split dataset into groups (Bundesland), count the members of each group ar
```

Out[60]:

Bundesland	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe
Bremen	15	15	15	15
Hamburg	19	19	19	19
Berlin	29	29	29	29
Saarland	54	54	54	54
Schleswig-Holstein	223	223	223	223
Mecklenburg-Vorpommern	237	237	237	237
Rheinland-Pfalz	264	264	264	264
Brandenburg	302	302	302	302
Sachsen	356	356	356	356
Thüringen	391	391	391	391
Sachsen-Anhalt	411	411	411	411
Hessen	436	436	436	436
Nordrhein-Westfalen	436	436	436	436
Niedersachsen	611	611	611	611
Baden-Württemberg	619	619	619	619
Bayern	1337	1337	1337	1337

In [108…
```
mask=station_ids.Bundesland=="Bayern" #when "Bayern" is true or false
```

In [109…
```
station_ids[mask]
```

Out[109…

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	B
10	Abbach,Bad-Dünzling	48.885697	12.107925	378.0	
15	Abenberg	49.234641	10.966676	391.5	
16	Abenberg-Wassermungenau	49.219251	10.884495	370.0	
17	Abensberg-Sandharlanden	48.842100	11.819366	380.0	
24	Achthal	47.833300	12.783300	582.0	
...	...	...	...	...	...
19694	Tittmoning/Oberbayern	48.066895	12.763446	382.0	
19695	Untergriesbach/Niederbayern	48.599508	13.646542	495.0	
19697	Wassertrüdingen/Wörnitz	49.046341	10.602684	435.0	
19701	Wertingen/Zusam	48.557500	10.674167	430.0	
19781	Ingolstadt	48.742909	11.423324	367.0	

1337 rows × 5 columns

In [105…
```
bayern=station_ids[mask] # start a new subgroup assigned as bayern
```

In [106…
```
station_ids.Bundesland.iloc[-1]
```

Out[106… 'Bayern'

In [107…
```
bayern.loc[17] #show position 17
```

Out[107…
```
Stationsname     Abensberg-Sandharlanden
geogr. Breite                    48.8421
geogr. Laenge                  11.819366
Stationshoehe                      380.0
Bundesland                         Bayern
Name: 17, dtype: object
```

Precipitation Data (exercise Alexanderplatz and follow up)

get a list of stations in Berlin
figure out the station ID of Berlin Alexanderplatz
load the 30 year mean precipitation data "data/Niederschlag 1981-2010.txt"
only keep the columns with the monthly means

In [82]:
```
precip_ref = pandas.read_csv("data/Niederschlag_1981-2010.txt", # get all pr
                             encoding="iso-8859-1", delimiter=" *; *",
                            index_col="Stations_id", engine="python")
```

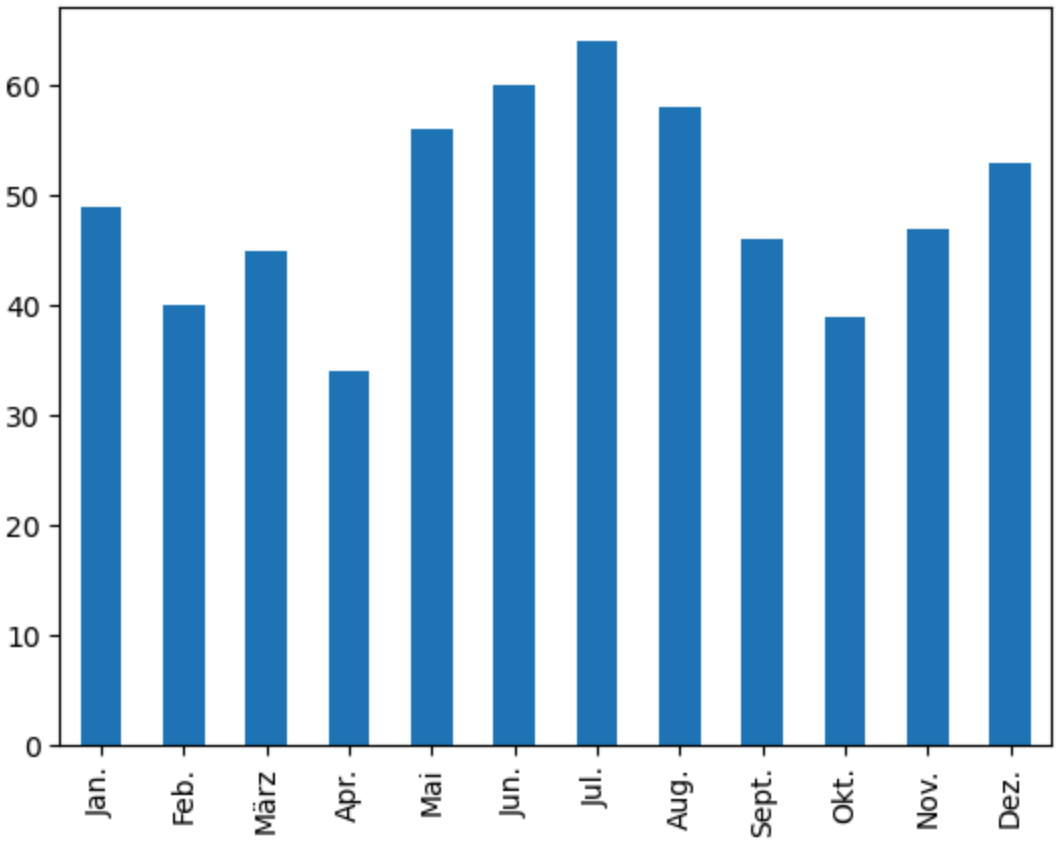
In [83]:
```
precip_ref = precip_ref.loc[:,"Jan.":"Dez."] #get monthly mean precipitatior
```

In [84]:
```
precip_ref.loc[399] # monthly mean precipitation on Berlin Alexanderplatz (1
```

Out[84]:
```
Jan.     49.0
Feb.     40.0
März     45.0
Apr.     34.0
Mai      56.0
Jun.     60.0
Jul.     64.0
Aug.     58.0
Sept.    46.0
Okt.     39.0
Nov.     47.0
Dez.     53.0
Name: 399, dtype: float64
```

In [85]:
```
precip_ref.loc[399].plot.bar()
```

Out[85]:
```
<Axes: >
```



In [73]:
```
precip = pandas.read_csv("data/produkt_precipitation_399_akt.txt",
                             encoding="iso-8859-1", delimiter=" *; *",
```

```
                                    engine="python")
```

In [74]:
```
precip.head()
```

Out[74]:

	STATIONS_ID	MESS_DATUM	QUALITAETS_NIVEAU	STRUKTUR_VERSION	NIE
0	399	2015082001	3	0	
1	399	2015082002	3	0	
2	399	2015082003	3	0	
3	399	2015082004	3	0	
4	399	2015082005	3	0	

In [76]:
```
precip["date_time"]=pandas.to_datetime(precip.MESS_DATUM, format="%Y%m%d%H")
#take the column and convert the numbers into the given data format (we over
```

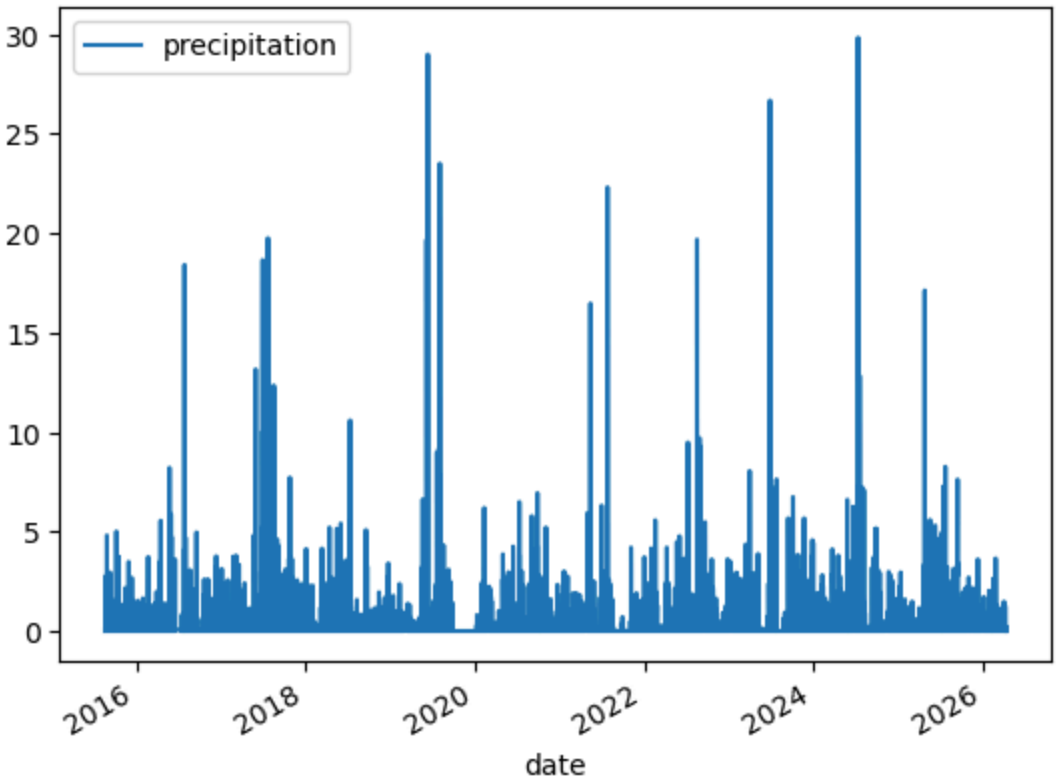
In [77]:
```
precip_alex=precip[["date_time", "NIEDERSCHLAGSHOEHE"]]
```

In [78]:
```
precip_alex=precip_alex.rename(columns={"date_time" : "date",
                                        "NIEDERSCHLAGSHOEHE" : "precipitatic
# rename the columns and overwrite the previous precip_alex
```

In [79]:
```
precip_alex=precip_alex.set_index("date")
```

In [80]:
```
precip_alex.plot()
```

Out[80]: <Axes: xlabel='date'>



In [101…
```
monthly = precip_alex.resample("ME").sum() # ME=monthly
```

In [102…
```
monthly.describe()
```

Out[102…

	precipitation
count	129.000000
mean	39.528372
std	28.812164
min	0.000000
25%	22.910000
50%	33.640000
75%	48.600000
max	166.300000

In [120…
```
precip_ref.loc[399]
```

Out[120…
```
Jan.     49.0
Feb.     40.0
März     45.0
Apr.     34.0
Mai      56.0
Jun.     60.0
Jul.     64.0
Aug.     58.0
Sept.    46.0
Okt.     39.0
Nov.     47.0
Dez.     53.0
Name: 399, dtype: float64
```

In [121…
```
monthly.groupby(monthly.index.month).describe() #for each year groups our da
```

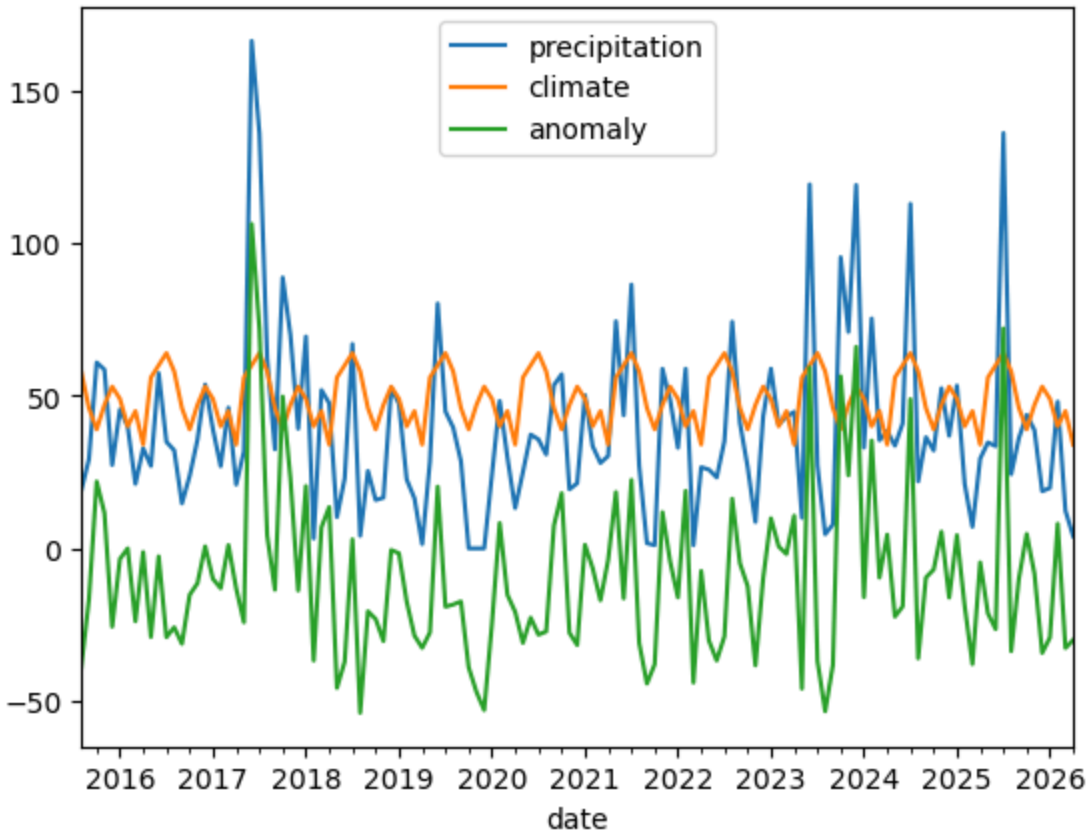
Out[121…

	precipitation								
	count	mean	std	min	25%	50%	75%	max	co
date									
1	11.0	43.080909	14.963813	19.80	33.0000	45.530	51.7750	69.42	1
2	11.0	38.052727	19.801915	3.24	24.7600	39.990	48.1950	75.33	1
3	11.0	26.577273	16.599816	1.02	14.3900	27.990	39.2900	51.87	1
4	11.0	26.310909	15.251537	1.41	17.0500	29.420	35.6550	47.56	1
5	10.0	30.115000	17.810316	10.03	25.2225	27.720	33.1750	74.44	1
6	10.0	62.480000	46.832982	22.91	34.4625	42.310	74.5300	166.30	1
7	10.0	71.625000	43.327513	27.16	35.3325	55.925	106.2875	136.10	1
8	11.0	30.906364	21.515000	4.12	20.4300	27.030	35.8800	74.27	1
9	11.0	27.925455	15.035998	1.70	20.0650	29.190	36.3100	53.48	1
10	11.0	40.480909	32.242088	0.00	19.8100	32.090	58.9950	95.41	1
11	11.0	39.063636	25.046306	0.00	17.9900	38.760	58.7200	70.93	1
12	11.0	41.870909	30.349011	0.00	24.3400	39.200	50.5300	119.06	1

12 rows × 24 columns

In [122…
```
monthly.plot()
```

Out[122…
```
<Axes: xlabel='date'>
```



In [123…
```
refs=pandas.DataFrame({"precipitation":precip_ref.loc[399].to_list()*12},
                      index=pandas.date_range("2015-01-01", "2026-12-31", fr
# create a new DataFrame with only one column: precipitation
# from climate data grab Alexanderplatz and convert it to a list, repeat thi
```

In [124…
```
refs
```

Out[124…

	precipitation
2015-01-31	49.0
2015-02-28	40.0
2015-03-31	45.0
2015-04-30	34.0
2015-05-31	56.0
…	…
2026-08-31	58.0
2026-09-30	46.0
2026-10-31	39.0
2026-11-30	47.0
2026-12-31	53.0

144 rows × 1 columns

In [125…
```
monthly["climate"] = refs # create a new column: climate
```

In [126…
```
monthly
```

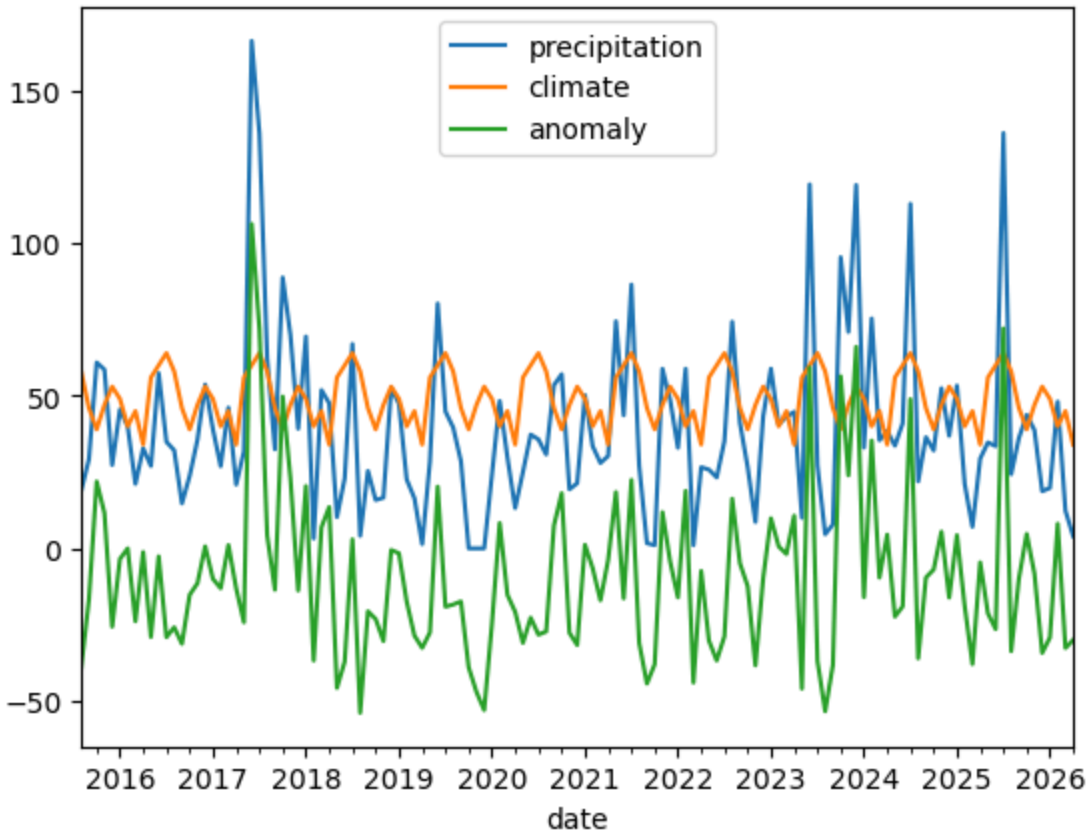
Out[126…

date	precipitation	climate	anomaly
2015-08-31	18.90	58.0	-39.10
2015-09-30	29.19	46.0	-16.81
2015-10-31	60.91	39.0	21.91
2015-11-30	58.59	47.0	11.59
2015-12-31	27.34	53.0	-25.66
…	…	…	…
2025-12-31	18.73	53.0	-34.27
2026-01-31	19.80	49.0	-29.20
2026-02-28	48.08	40.0	8.08
2026-03-31	12.36	45.0	-32.64
2026-04-30	4.00	34.0	-30.00

129 rows × 3 columns

In [127…
```
monthly.plot()
```

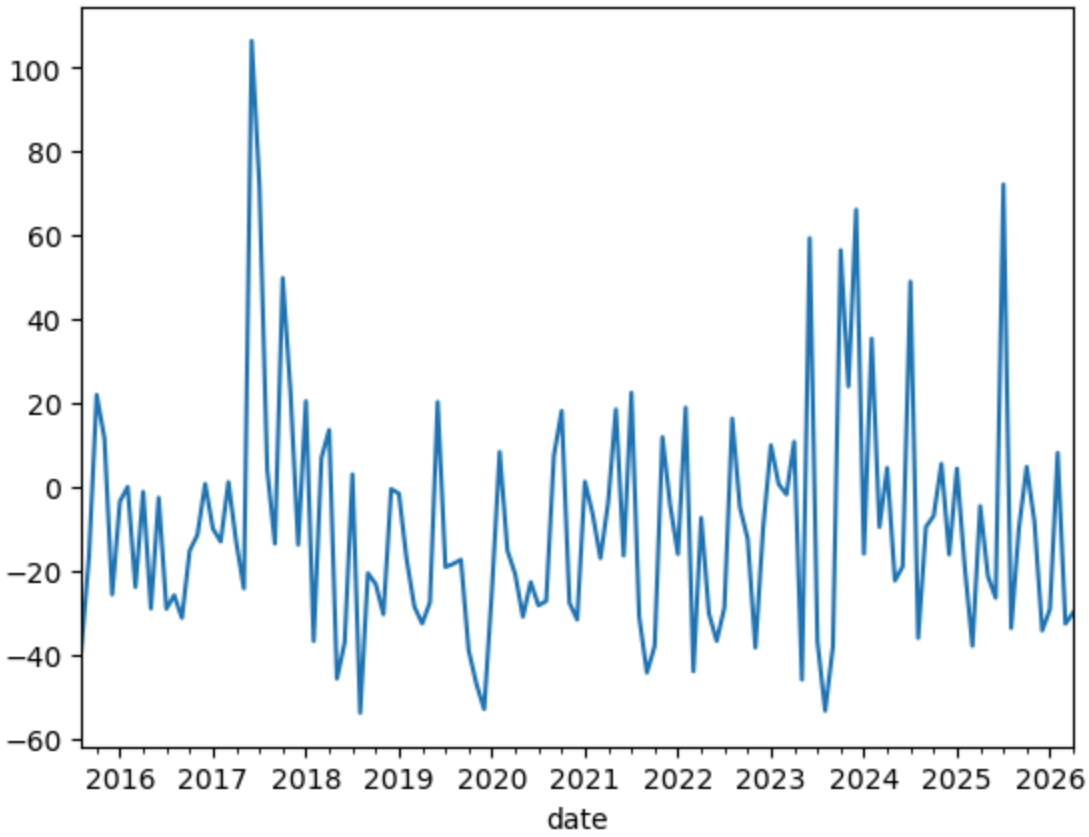
Out[127…
```
<Axes: xlabel='date'>
```



In [128…
```
monthly["anomaly"]=monthly.precipitation - monthly.climate
#to just get the defference between the general mean precipitation and the m
```

In [119…
```
monthly.anomaly.plot()
# many spikes in minus-area, so Berlin is dryer
```

Out[119… <Axes: xlabel='date'>


100
80
60
40
20
0
−20
−40
−60
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
date

In [ ]: